Implementing a Poisson Equation Solver using C++ and CUDA, using a Multigrid V-Cycle approach, with optimisation using Nsight

Karthigan Sabanadesan

October 11, 2026

Abstract

This document describes my recent project of creating a Poisson solver in CUDA C++, along with my efforts in optimising its computational efficiency. This project was chosen because it combines the practical challenge of GPU programming with the opportunity to improve the performance of a computationally demanding solver. It would allow me to work on complex solvers, challenging me to learn how to translate unfamiliar mathematics into effective and efficient solver, and also be one that works for range of test solutions and boundary conditions. The document details my design considerations, testing procedure, and performance analysis.

1 Introduction

1.1 Poisson's Equation Background

The Poisson equation is an elliptic partial differential equation that arises in a wide range of physical contexts, including Newtonian gravity, electrostatics, and fluid dynamics [Hun]. Thus it is of considerable interest to those who work with complex problems in physics, requiring numerical solvers to obtain practical working solutions because many problems in these areas do not admit simple closed-form solutions [PTVF07]. The general equation takes the form in three-dimensional Cartesian coordinates:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = f(x, y, z), \text{ where } u = u(x, y, z) \text{ and } (x, y, z) \in \Omega \tag{1}$$

When $f(x, y, z) = 0$, Poisson's equation reduces to Laplace's equation, $\nabla^2 u = 0$ for $(x, y, z) \in \Omega$. Poisson's equation may therefore be regarded as the inhomogeneous, or forced, version of Laplace's equation, where the source term f represents an external influence on the field. A well-posed problem also requires boundary conditions on $\partial\Omega$. For partial differential equations, Dirichlet boundaries take the form [AWH13]:

$$u(\mathbf{x}) = g_D(\mathbf{x}), \quad \forall\ \mathbf{x} \in \partial\Omega$$

Other common choices are Neumann conditions, which prescribe the normal derivative [AWH13]:

$$\frac{\partial u}{\partial n} = g_N(\mathbf{x}), \qquad \forall\ \mathbf{x} \in \partial\Omega.$$

Robin conditions, which prescribe a linear combination of the solution and its normal derivative[AWH13]:

$$\frac{\partial u}{\partial n} + \alpha u = g_R(\mathbf{x}), \qquad \mathbf{x} \in \partial\Omega.$$

1.2 Discrete Formulation

The continuous problem is discretised on a sequence of nested uniform 3D grids. If the finest grid has $N = 2^k + 1$ grid points per side over a domain of characteristic length L, the grid spacing is:

$$h = \frac{L}{N - 1}. \tag{2}$$

The power of grid size allows successive coarsening by a factor of two until the coarsest grid level is reached. By using a standard seven-point finite-difference approximation, the discrete problem can be written as:

$$Au = f, \tag{3}$$

where A is the discrete Laplacian, u is the vector of unknown grid values, and f is the vector of source-term values. If the solver uses the positive-definite sign convention:

$$-\nabla^2 u \;=\; f, \tag{4}$$

then the 3D seven-point stencil is:

$$(Au)_{i,j,k} = \frac{6u_{i,j,k} - u_{i+1,j,k} - u_{i-1,j,k} - u_{i,j+1,k} - u_{i,j-1,k} - u_{i,j,k+1} - u_{i,j,k-1}}{h^2} = f_{i,j,k}, \tag{5}$$

and the discrete residual is:

$$r = f - Au. \tag{6}$$

The solver uses a geometric V-cycle multigrid approach, where, at each grid level, the cycle consists of the following steps:
1. **Pre-smoothing:** Apply ν_1 iterations of a red-black Gauss-Seidel (or Jacobi) smoother on the fine grid, to reduce high-frequency components of the error on the domain. In 3D, red-black ordering can be defined by the parity of $i + j + k$.
2. **Residual computation:** Compute the residual: $r = f - Au$.
3. **Restriction:** Transfer the fine-grid residual to the next coarser grid using a full-weighting restriction operator.
4. **Coarse-grid correction:** Recursively call the V-cycle on the next coarser level (e.g., do a direct solve on the coarsest grid, or a few Jacobi iterations).
5. **Prolongation:** Perform trilinear interpolation of the coarse-grid correction onto the fine grid.
6. **Post-smoothing:** Apply ν_2 iterations of the same smoother onto the fine grid.
For Dirichlet boundary conditions, values are kept fixed; they are simply never updated. In practice, boundary unknowns may either be excluded from the update or the boundary values may be re-imposed after each smoothing, restriction, and prolongation step [BHM00].

1.3 Red–Black Gauss–Seidel Smoothing

Red–black Gauss–Seidel is well suited to execution on parallel architectures such as GPUs because it removes data dependencies within each colour phase [KHEH22, Saa03]. The three-dimensional grid is partitioned into red and black points using the chequerboard parity:

$$(i + j + k) \bmod 2.$$

When updating the red points, all of their immediate neighbours are black points, and those black values are not modified during the red phase. The black points are then updated using the newly computed red values. A synchronisation barrier is therefore required between the red and black phases to ensure that the updated values are visible before the next phase begins.
For a uniform grid with spacing h and for the continuous problem:

$$\nabla^2 u = f,$$

the Gauss–Seidel update at an interior grid point is:

$$u_{i,j,k}^{\text{new}} = \frac{1}{6}\left[u_{i+1,j,k} + u_{i-1,j,k} + u_{i,j+1,k} + u_{i,j-1,k} + u_{i,j,k+1} + u_{i,j,k-1} - h^2 f_{i,j,k}\right]. \tag{7}$$

If instead the formulation uses the sign convention:

$$-\nabla^2 u = f,$$

then the source-term contribution changes sign:

$$u_{i,j,k}^{\text{new}} = \frac{1}{6}\left[u_{i+1,j,k} + u_{i-1,j,k} + u_{i,j+1,k} + u_{i,j-1,k} + u_{i,j,k+1} + u_{i,j,k-1} + h^2 f_{i,j,k}\right]. \tag{8}$$

Since this solver explicitly implements the positive-definite convention ($-\nabla^2 u = f$), the update step follows Equation (6). The iterative procedure may be summarised as follows:

1. Initialise the interior values $u_{i,j,k}$ with an initial guess.
2. Impose the physical Dirichlet boundary values along the finest grid boundary $\partial\Omega$.
3. Update all red interior points in parallel.
4. Synchronise by executing a thread synchronization barrier.
5. Update all black interior points in parallel.
6. Synchronise by executing a thread synchronization barrier.
7. Evaluate the residual vector and check convergence metrics periodically.
8. Repeat until the relative residual falls below a prescribed tolerance.

1.4 Solvers and Convergence Metrics

The convergence of the iterative solver is measured using the residual:

$$r^{(m)} = f - Au^{(m)},$$

where A is the discrete Poisson operator and $u^{(m)}$ is the numerical solution after m iterations. The discrete L_2 residual norm is:

$$\|r^{(m)}\|_2 = \left(h^3 \sum_{i,j,k} \left| r_{i,j,k}^{(m)} \right|^2 \right)^{1/2}.$$

In practice, the relative residual:

$$\frac{\|r^{(m)}\|_2}{\|f\|_2}$$

is used as the convergence measure. The iteration is terminated when this quantity falls below a prescribed tolerance.
For validation against the manufactured analytical solutions, the numerical error is defined as:

$$e_{i,j,k}^{(m)} = u_{i,j,k}^{(m)} - u_{\text{exact}}(p_{i,j,k}),$$

where $p_{i,j,k}$ is the physical coordinate of grid point (i, j, k) [OR10]. The discrete L_2 error norm is:

$$\|e^{(m)}\|_2 = \left(h^3 \sum_{i,j,k} \left| e_{i,j,k}^{(m)} \right|^2 \right)^{1/2},$$

and the maximum-error norm is:

$$\|e^{(m)}\|_\infty = \max_{i,j,k} \left| e_{i,j,k}^{(m)} \right|.$$

The L_2 norm provides a global measure of accuracy because the squared errors are accumulated over the whole domain. The L_∞ norm is also useful because it highlights the largest local error in the solution.

1.5 Language choice

CUDA C++ was chosen because it provides direct control over GPU memory and execution while requiring a relatively low learning curve for a programmer already familiar with C++. It exposes the hardware features most relevant to high-performance solvers, including shared memory, atomic operations, and explicit control over thread and block organisation [NVI26a]. The project also allowed me to develop practical GPU programming skills and to investigate how these features affect numerical performance.
CUDA C++ minimises unnecessary abstraction overhead, as the NVIDIA `nvcc` compiler builds host and device code into a single executable, allowing kernels to be launched directly from C++ host code.

This avoids some of the overhead associated with calling GPU code from high-level interpreted languages, where data movement across language boundaries can introduce marshalling and performance penalties. CUDA C++ permits the manual management of Shared Memory (the on-chip cache shared among threads in a block), and allows use of dedicated optimisations of the `nvcc` compiler for code compilation in C++ [NVI26b].
CUDA C++ also provides access to a mature scientific-computing ecosystem. Optimised libraries such as cuBLAS, cuFFT, Thrust, and CUB supply common numerical and parallel primitives, reducing the need to implement components such as matrix multiplication, fast Fourier transforms, and sorting routines from scratch [NVI26i]. Finally, CUDA is widely adopted in academic and industrial high-performance computing environments, providing a large body of literature, examples, and community support.

2 Key Architectural Highlights

2.1 Code Structure

The three-dimensional computational domains are stored as Linearised 3D arrays (in the form x + y * Width + z * Width * Height) for our 3 Dimensional domain, because physical GPU memory is organised as a single contiguous 1D ribbon of bytes. A grid point (x, y, z) is mapped to a linear index:

$$I(x, y, z) = x + y * N_x + z * N_x * N_y \tag{9}$$

where N_x and N_y are the numbers of grid points in the x- and y-directions, respectively. This row-major layout ensures that adjacent cells in the fastest-varying x-direction are physically adjacent in device memory. Global memory on a GPU is fetched in aligned memory transactions (also referred to as chunks) of 32-byte, 64-byte, or 128-byte sectors, so we want to ensure that we effectively utilize these characteristics by compacting our grid values as closely as possible for adjacent cells in every direction. When a group of 32 threads (a warp) executes a memory instruction, the GPU hardware attempts to combine those 32 separate reads into one or more hardware memory transaction — a process called memory coalescing [WL, NVI26a].
For the three-dimensional seven-point Poisson stencil, each thread reads the centre point and its six axis-aligned neighbours:

$$u_{i\pm1,j,k}, \qquad u_{i,j\pm1,k}, \qquad u_{i,j,k\pm1}.$$

If the threads in a warp are mapped to consecutive x-coordinates, the address streams for these accesses are spatially compact. This allows the GPU to coalesce the requests more effectively and reduces the number of memory transactions required per stencil evaluation.
If we use nested pointers (arr[z][y][x]), rows and slices can be spread around, to non-contiguous blocks across the physical VRAM and cause extra pointer dereferencing. Accessing a single value would require dereferencing the z-slice pointer, then the y-row pointer, before finally accessing the value at the x-index. These dependent loads increase memory latency and can fragment the effective memory access pattern, reducing coalescing and hardware utilisation. This causes "strided access", which drastically reduces hardware utilization across the GPU, as well as increasing memory latency. We will be struggling with low hardware utilization in the smaller grids, so we don't want to compound this issue further.
Flattening the array turns this into a single mathematical index calculation (ptr[index]), allowing the GPU to fetch the data in a single instruction. By linearizing the 3D grid into row-major order, you guarantee that adjacent indices in space (e.g., cell (x, y, z) and cell $(x + 1, y, z)$) are physically adjacent in the VRAM hardware chip. Threads mapped to consecutive X coordinates can pull data simultaneously in one clock cycle, reducing the number of overall transactions required.

2.2 Headers

2.2.1 General C++ Headers

List of C++ headers used in the program, with a short description of their function.

- `<iostream>` — For standard input and output streams.

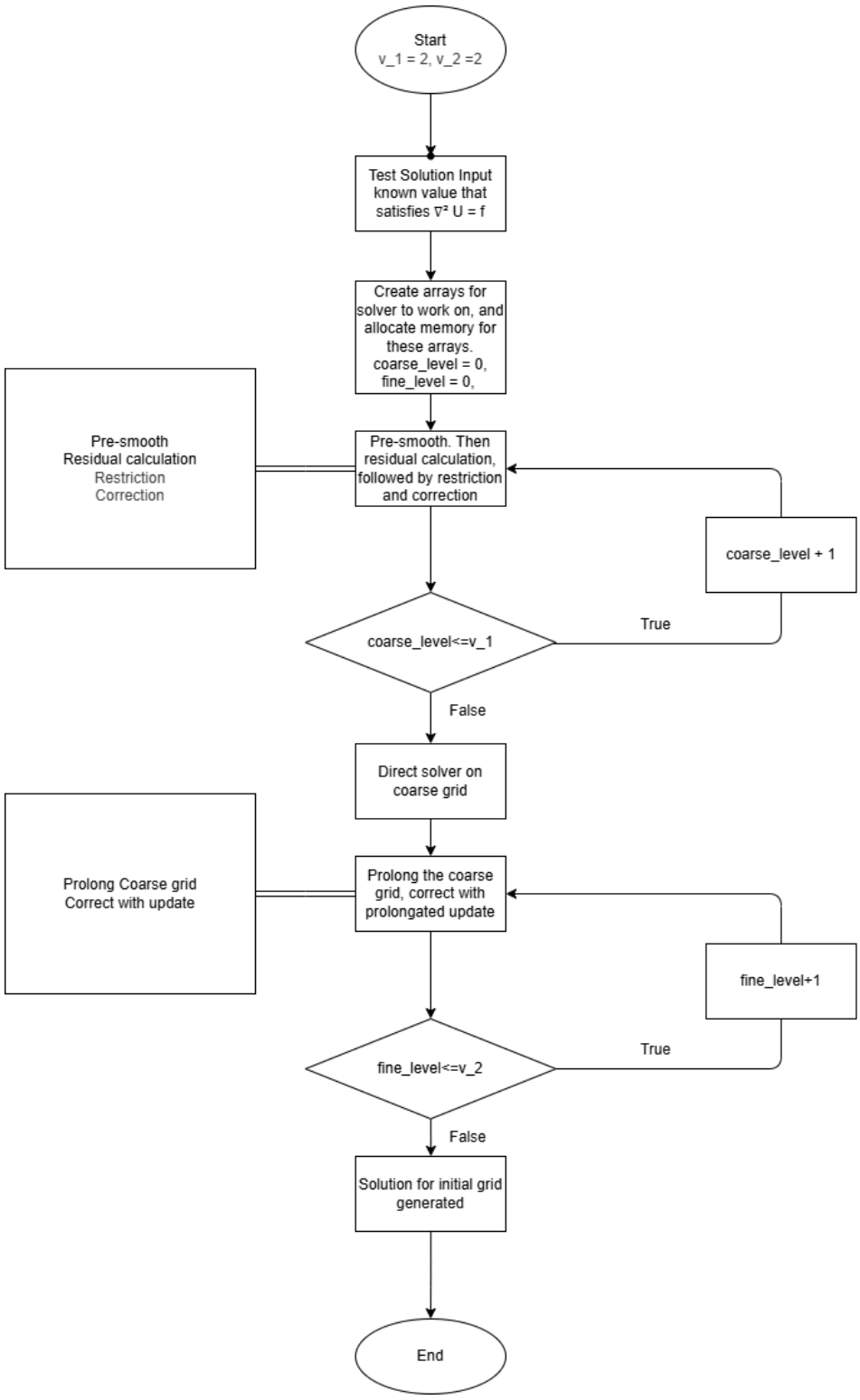


Figure 1: Flowchart of Poisson Solver. This is particularly helpful when it comes to structuring the `main()` function, as it show clearly the interactions between the `main()` and its helper functions

- `<fstream>` — Read from and write to external files
- `<vector>` — Allows use of `std::vector`, the standard C++ dynamically resizing array container, which handles contiguous memory management automatically.
- `<sstream>` — Introduces `std::stringstream`, allowing you to treat a `std::string` as an I/O stream.
- `<iomanip>` — Allows the formatting of console and stream output, such as setting decimal precision (`std::setprecision`) or establishing column widths (`std::setw`).
- `<cmath>` — C++ wrapper for the standard C math library. Offers basic mathematical functions like `std::sqrt()`, `std::pow()`, `std::sin()`, and `std::abs()`.
- `<cstdio>` — C++ wrapper for the traditional C I/O library (`<stdio.h>`). It brings in C-style functions like `printf()` and `scanf()`.
- `<cstdint>` — Defines fixed-width integer types, such as int32_t, uint64_t). Guarantees consistent byte sizes across different hardware architectures.
- `<chrono>` — A robust time utility library used for measuring durations, working with clocks, and timing how long a specific block of code takes to execute.

2.2.2 CUDA Specific Headers

- `<cuda_runtime.h>` — Primary header for the CUDA Runtime API. Grants access to essential GPU operations including device memory allocation (`cudaMalloc`), host-to-device memory transfers (`cudaMemcpy`), and device management.
- `<device_launch_parameters.h>` — Automatically manages internal GPU launch variables. It defines vital spatial variables like built-in variables `gridDim`, `blockDim`, `blockIdx`, and `threadIdx` used to map execution coordinates inside a GPU kernel.
- `<nvtx3/nvtx3.hpp>` — NVIDIA Tools Extension (NVTX) library. Allows you to inject custom markers, time ranges, and names into your code. These annotations appear directly inside profiling timelines like NVIDIA Nsight Systems to ease performance debugging [NVI26k]. Crucial for any rigorous performance analysis of the code.
- `<cub/cub.cuh>` — Stands for CUDA Unbound (CUB). Provides optimized, block-wide, cooperative parallel primitives (such as sorting, reductions, and prefix sums) designed specifically for writing highly performant custom CUDA kernels [MN24].
- `<cuda/functional>` — Provides a GPU-compatible implementation of functional programming tools (similar to the standard `<functional>`), enabling the use of features like `cuda::std::plus` or custom function objects inside device code.
- `<thrust/iterator/zip_iterator.h>` — Part of Thrust, a high-level parallel algorithms library. A zip_iterator lets you bind multiple separate arrays into a single virtual array of tuples, allowing parallel algorithms to process multiple arrays simultaneously in a single pass [BH12].
- `<thrust/tuple.h>` — GPU-compatible implementation of a tuple (a finite, ordered sequence of elements or values), which is required to pass bundled data structures through Thrust parallel iterators and algorithms [BH12].

2.3 Main Kernels and Functions

2.3.1 `red_black_gauss_siedel_red()` and `red_black_gauss_siedel_black()`

The original red–black Gauss–Seidel implementation uses two CUDA kernels, `red_black_gauss_siedel_red()` and `red_black_gauss_siedel_black()`. The two kernels are identical except for the chequerboard parity test. Each kernel assigns one CUDA thread to one grid point (x, y, z). Boundary threads return immediately, so only interior grid points are updated.

The red kernel updates cells for which $x+y+z$ is even, while the black kernel updates cells for which $x+y+z$ is odd. For an interior point i, the kernel forms the sum of the six face-neighbour values:

$$S_i = u_{i-1} + u_{i+1} + u_{i-N_x} + u_{i+N_x} + u_{i-N_xN_y} + u_{i+N_xN_y},$$

and performs the in-place update:

$$u_i \leftarrow \frac{1}{6}\left(S_i + h^2 f_i\right).$$

This is the red–black Gauss–Seidel half-sweep for the negative Laplacian $-\nabla^2 u = f$ with unit relaxation parameter. Because the six face neighbours of a red cell are black, all red cells can be updated in parallel. The black kernel is then launched to update the opposite colour using the newly updated red values.

2.3.2 `InitializeProblem()`

The `InitializeProblem()` kernel initialises the unknown solution, source term, and reference solution. It converts discrete integer grid coordinates (x, y, z) into continuous physical coordinates (p_x, p_y, p_z) using the grid spacing. For a uniform grid with spacing h,

$$p_x = \Delta x = xh, \qquad p_y = \Delta y = yh, \qquad p_z = \Delta z = zh.$$

The pointers to the solution array `u`, the source array `f`, and the reference array `u_true` are declared using `__restrict__`. This informs the compiler that the pointer regions do not overlap in memory, allowing for additional optimisations for optimising memory usage and maximise cache efficiency.
Because consecutive threads in a warp are assigned consecutive x-indices, the writes to `u[i]`, `f[i]`, and `u_true[i]` are coalesced. The GPU memory system can therefore combine these writes into efficient memory transactions, improving effective memory bandwidth.

2.3.3 `Residual_function()`

The initial implementation of `Residual_function()` assigned one CUDA thread to each grid point. The thread coordinates (x, y, z) were obtained from the CUDA block and thread indices and converted to a linear index i using the helper function `getLinearIdx3D()`. Threads corresponding to boundary grid points returned before evaluating the stencil. For an interior point i, the kernel computed:

$$L_i = -\Big(u_{i-1} + u_{i+1} + u_{i-N_x} + u_{i+N_x} + u_{i-N_xN_y} + u_{i+N_xN_y} - 6u_i\Big)h^{-2},$$

and stored:

$$r_i = f_i - L_i.$$

The quantity L_i is algebraically equivalent to:

$$L_i = \frac{6u_i - \sum_{j\in\mathcal{N}(i)} u_j}{h^2},$$

where:

$$\mathcal{N}(i) = \{i-1,\, i+1,\, i-N_x,\, i+N_x,\, i-N_xN_y,\, i+N_xN_y\}.$$

Thus the kernel computes the residual:

$$r_i = f_i - Au_i,$$

with:

$$Au_i = \frac{6u_i - \sum_{j\in\mathcal{N}(i)} u_j}{h^2}.$$

The original kernel used ordinary global-memory loads for f and u, accepted both h and h^2 as arguments even though only h^2 was used, and computed the linear index through a helper function.

2.3.4 `Coarsen()`

The kernel `Coarsen()` restricts the fine-grid solution to a coarse grid using factor-of-two coarsening. Each CUDA thread is assigned to one coarse-grid point (x, y, z) and maps to the corresponding fine-grid centre $(2x, 2y, 2z)$. The coarse value is computed using a 27-point weighted stencil centred at that fine-grid point:

$$R_i = \sum_{d_x,d_y,d_z\in\{-1,0,1\}} w(d_x, d_y, d_z)u_f(2x + d_x,\, 2y + d_y,\, 2z + d_z),$$

where the weight is:

$$w(d_x, d_y, d_z) = 2^{-(3+|d_x|+|d_y|+|d_z|)}.$$

Thus the centre point has weight 1/8, face neighbours have weight 1/16, edge neighbours have weight 1/32, and corner neighbours have weight 1/64. The weights sum to one, so the restriction is a normalized weighted average.

Near the domain boundary, out-of-range fine-grid indices are clamped to the boundary, so missing stencil points reuse the nearest boundary value. The resulting coarse-grid value is written to the coarse array at the corresponding coarse-grid index.

2.3.5 `ProlongAndCorrect()`

The kernel `ProlongAndCorrect` prolongates the coarse-grid error to the fine grid and adds the interpolated correction to the fine-grid solution. It assumes a regular 2:1 grid refinement in each spatial direction. Each CUDA thread is responsible for one fine-grid point (f_x, f_y, f_z). Threads corresponding to physical boundary points return without performing arithmetic, so only interior fine-grid points are corrected.

For an interior fine-grid point, the kernel determines the enclosing coarse-grid cell by

$$c_x = \left\lfloor \frac{f_x}{2} \right\rfloor, \quad c_y = \left\lfloor \frac{f_y}{2} \right\rfloor, \quad c_z = \left\lfloor \frac{f_z}{2} \right\rfloor.$$

The upper coarse-grid indices are clamped to the last valid coarse-grid index, for example:

$$c_x^+ = \min(c_x + 1, N_x^c - 1),$$

with analogous definitions for c_y^+ and c_z^+. This prevents out-of-bounds access when the fine-grid point is associated with the last coarse-grid cell. The local interpolation weights are:

$$t_x = \frac{f_x \bmod 2}{2}, \qquad t_y = \frac{f_y \bmod 2}{2}, \qquad t_z = \frac{f_z \bmod 2}{2}.$$

Thus, a fine-grid point located on a coarse-grid node has $t = 0$, while a fine-grid point located halfway between coarse-grid nodes has $t = 1/2$. Let E^c denote the coarse-grid error array. The eight surrounding coarse-grid values are written as:

$$e_{abc} = E^c(c_x + a,\ c_y + b,\ c_z + c), \qquad a, b, c \in \{0, 1\},$$

where the coarse-grid indices are clamped to the valid coarse-grid range. The kernel tri-linearly interpolates these values to the fine-grid location:

$$\begin{aligned} p_f = (1 - t_z)\Big[(1 - t_y)\big((1 - t_x)e_{000} + t_x e_{100}\big) + t_y\big((1 - t_x)e_{010} + t_x e_{110}\big)\Big] \\ + t_z\Big[(1 - t_y)\big((1 - t_x)e_{001} + t_x e_{101}\big) + t_y\big((1 - t_x)e_{011} + t_x e_{111}\big)\Big]. \end{aligned}$$

The interpolated correction p_f is then added directly to the fine-grid solution:

$$u_f \leftarrow u_f + p_f.$$

Because each thread writes to a distinct fine-grid location, no additional synchronization is required.

2.3.6 `v_cycle_hierarchical()`

Recursive function that calls `Residual_function()`, `Coarsen()`, `ProlongAndCorrect()`, `red_black_gauss_siedel_black()` and `red_black_gauss_siedel_red()`. It functions as the linchpin of the program. The host routine `v_cycle_hierarchical` drives one recursive geometric V-cycle over the multigrid hierarchy. At level ℓ, it uses the solution u_ℓ, right-hand side f_ℓ, residual vector r_ℓ, and grid dimensions $(N_x^\ell, N_y^\ell, N_z^\ell)$. Each CUDA kernel launch uses a fixed thread-block configuration and a grid sized for the active level.
The routine first applies ν_1 pre-smoothing cycles. Each cycle consists of one red and one black Gauss–Seidel sweep. If ℓ is the coarsest level, the routine applies ten additional smoothing cycles and returns, using the result as the approximate coarse-grid correction. Otherwise, the residual:

$$r_\ell = f_\ell - A_\ell u_\ell$$

is computed on the current level. The coarse-grid solution is initialized to zero, and the residual is restricted to the coarse right-hand side:

$$f_{\ell+1} = R_\ell r_\ell, \qquad u_{\ell+1} = 0.$$

The routine then recursively calls itself on level $\ell + 1$, where the grid spacing is doubled, $h_{\ell+1} = 2h_\ell$. After the recursive call returns, the coarse-grid correction $u_{\ell+1}$ is prolongated to the fine grid and added to the current fine-grid solution:

$$u_\ell \leftarrow u_\ell + P_\ell u_{\ell+1}.$$

Finally, ν_2 post-smoothing cycles are applied on level ℓ.

2.3.7 `main()`

The main program sets up and executes a manufactured-solution benchmark for the 3D multigrid solver. The finest grid is a cubic grid of size $N = 1025$ in each direction on the unit domain, giving a grid spacing:

$$h = \frac{1}{N-1}.$$

A multigrid hierarchy is constructed dynamically using vertex-centred coarsening:

$$N_{\ell+1} = \left\lfloor \frac{N_\ell - 1}{2} \right\rfloor + 1,$$

until the coarsest grid size reaches 33. For the default finest-grid size $N = 1025$, this produces six hierarchy levels:

$$1025^3,\ 513^3,\ 257^3,\ 129^3,\ 65^3,\ 33^3.$$

For each level, device arrays are allocated and initialised to zero for the solution u, the right-hand side f, and the residual r.
The finest-level right-hand side and manufactured solution are generated by the `InitializeProblem` kernel. The initial fine-grid solution is zero. The solver then performs five V-cycles, each starting from the finest level with two pre-smoothing and two post-smoothing iterations. After each V-cycle, the root-mean-square error between the numerical fine-grid solution and the manufactured solution is computed and reported. NVTX markers are inserted around the major execution stages to support timeline analysis. Finally, all device memory is released.

2.3.8 Utility functions and macros

Several helper functions, macros, and functors were implemented to simplify the CUDA code and improve numerical robustness.

- `CUDA_CHECK` is a macro used to check the return values of CUDA API calls. If a CUDA call fails, the macro prints the source file and line number where the error occurred, converts the numeric error code into a readable message, and terminates the program. This makes it easier to locate errors such as illegal memory accesses or initialisation failures.

- `getLinearIdx3D()` maps a three-dimensional index (x, y, z) to a one-dimensional linear index:

$$\text{idx} = (\texttt{size_t})z \cdot N_y \cdot N_x + (\texttt{size_t})y \cdot N_x + x,$$

The explicit casts to `size_t` promote the multiplication to 64-bit arithmetic and reduce the risk of integer overflow for large grids. An equivalent form is

$$\text{idx} = (((\texttt{size_t})z \cdot N_y + y) \cdot N_x + x),$$

which casts only the leading term before promoting the subsequent arithmetic to 64 bits.

3 Testing

3.1 Execution environment

The solver was developed and tested inside a Docker container running on WSL2 on a Windows 11 host. This setup provided a reproducible development environment and avoided manual configuration of CUDA tools and environment variables in the host operating system. The base image was the official NVIDIA CUDA development image for Ubuntu 22.04 LTS:`nvidia/cuda:13.3.0-devel-ubuntu22.04`. This image includes the CUDA compiler toolchain, debugging tools, and required headers. The container was built and started using the following commands:

```
docker build -t cuda133-cpp-dev .
docker run --gpus all -it -v ${PWD}:/workspace cuda133-cpp-dev bash
```

To confirm that the host NVIDIA driver and Docker container runtime were correctly configured, the following commands were executed:

```
nvidia-smi
docker --version
nvidia-ctk runtime configure --runtime=docker
systemctl restart docker
```

Nsight Systems was then installed inside the container to profile program execution:

```
apt-get update
apt-get install -y cuda-nsight-systems-13-3
nsys --version
```

For system-level profiling, the executable was run with:

```
nsys profile --trace=cuda,nvtx -o poisson_report ./Poisson_solver
```

For kernel-level profiling, Nsight Compute was used with commands of the form:

```
ncu --set full -o Poisson_Redesign_opt ./Poisson_solver_redesign_opt_initialised
```

Because the container does not provide a graphical display, the generated reports were opened in the Nsight Systems GUI on Windows. This allowed the GPU execution timeline, kernel launches, and memory operations to be inspected as both a timeline and a tabular summary. This workflow combined the convenience of a containerised build environment with full access to the NVIDIA profiling tools.

3.2 Hardware and software

The hardware platform was a Palit RTX 5090 32 GB graphics card based on the NVIDIA GB202 GPU. The GPU implements the Blackwell architecture and supports CUDA compute capability 12.0. It contains 170 streaming multiprocessors (SMs), 21760 CUDA cores, and 680 Tensor cores. Each SM contains up to 128 KB of L1 cache, and the GPU has 96 MB of L2 cache. Memory is provided by 32 GB of GDDR7 over a 512-bit bus, giving a memory bandwidth of approximately 1.79 TB/s. The theoretical peak performance is approximately 104.8 TFLOP/s for FP16 and FP32, and 1.637 TFLOP/s for FP64. The GPU is connected to the host through a PCIe 5.0 x16 interface, providing

64 GB/s unidirectional and 128 GB/s bidirectional bandwidth.
The host system ran Windows 11 25H2, build 26200.8655, with an AMD Ryzen 9 9800X3D processor and 96 GB of SK Hynix M-Die DDR5 memory at 6000 MHz. Because the solver is GPU-centric, the CPU primarily handled kernel launches, host–device data transfers, and timing overhead. The large system memory capacity ensured that host memory would not be a limiting factor.
The software environment used CUDA Toolkit 13.3, specifically V13.3.33, build `cuda_13.3.r13.3/compiler.37862127_0`, with NVIDIA driver version 610.88. The implementation used the C++17 standard and was compiled with NVCC 13.3. The Thrust library was used for parallel data structures and algorithms. Profiling was performed using NVIDIA Nsight Compute and Nsight Systems to identify performance bottlenecks.
Additional performance considerations included comparing theoretical and achieved occupancy, using shared memory to reduce global memory traffic, structuring conditional code to minimise warp divergence, and using asynchronous CUDA streams to overlap host–device transfers with kernel execution where appropriate.

3.3 Test Solutions

A set of manufactured solutions is used to verify the solver. For each test case, an exact solution $u_{\text{exact}}(x, y, z)$ is chosen analytically, and the corresponding source term is defined by

$$f = \nabla^2 u_{\text{exact}}.$$

The numerical solution is then compared with the exact solution on the unit cube

$$\Omega = [0, 1]^3.$$

Test Case 1: Triple-sine solution with zero Dirichlet boundaries
The first test case is:

$$u_{\text{exact}}(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z).$$

The corresponding source term is:

$$f(x, y, z) = -3\pi^2 \sin(\pi x) \sin(\pi y) \sin(\pi z).$$

The boundary condition is homogeneous Dirichlet:

$$u = 0 \ \forall \ (x, y, z) \in \partial\Omega.$$

This is a standard test problem for three-dimensional geometric multigrid. Because the boundary values are zero, it is particularly useful for checking the interior seven-point stencil and the smoothing properties of the method.

Test Case 2: Polynomial solution with non-zero Dirichlet boundaries
The second test case uses the cubic polynomial:

$$u_{\text{exact}}(x, y, z) = x^3 + y^3 + z^3.$$

The corresponding source term is:

$$f(x, y, z) = 6x + 6y + 6z.$$

The boundary values are non-zero and are obtained by evaluating u_{exact} on $\partial\Omega$. For example:

$$u(0, y, z) = y^3 + z^3,$$
$$u(1, y, z) = 1 + y^3 + z^3,$$

with analogous expressions on the remaining y- and z-faces.

This case verifies that non-zero Dirichlet boundary values are correctly incorporated into the discrete residual and into the multigrid transfer operators.

Test Case 3: Mixed-frequency trigonometric solution
The third test case is:

$$u_{\text{exact}}(x, y, z) = \sin(2\pi x)\cos(\pi y)\sin(3\pi z).$$

The corresponding source term is:

$$f(x, y, z) = -14\pi^2 \sin(2\pi x)\cos(\pi y)\sin(3\pi z).$$

The boundary values are evaluated explicitly from u_{exact}. Since $\sin(2\pi x) = 0$ at $x = 0$ and $x = 1$, and $\sin(3\pi z) = 0$ at $z = 0$ and $z = 1$, $u = 0$ at $x = 0,\ x = 1,\ z = 0,\ z = 1$. On the y-faces,

$$u(x, 0, z) = \sin(2\pi x)\sin(3\pi z),$$
$$u(x, 1, z) = -\sin(2\pi x)\sin(3\pi z).$$

This solution contains multiple wave numbers and therefore tests the solver over a broader frequency spectrum.

Test Case 4: Exponential solution
The fourth test case is:

$$u_{\text{exact}}(x, y, z) = e^{x+2y+3z}.$$

The corresponding source term is:

$$f(x, y, z) = 14e^{x+2y+3z}.$$

Dirichlet boundary conditions are applied on all six faces of the unit cube by setting:

$$u = u_{\text{exact}} \qquad \text{on } \partial\Omega.$$

This case introduces a smooth, non-symmetric solution with different decay rates in each coordinate direction. It is useful for checking that the discrete stencil and multigrid operators do not introduce directional bias. For each test case, a CUDA initialisation kernel populates the source term f, the initial solution u, and the reference solution u_{exact} directly in device memory. This allows the test problem to be passed directly to the solver without additional intermediate data transfer. To test these functions, a kernel was created to initialise the problem for in CUDA C++, so that it could be fed directly into the solver. And so we have the same numbers generated each time the program is run, thus it is deterministic.

4 Performance analysis using CUDA specific tools

4.1 The 'Speed of Light' (SOL)

A useful baseline for performance analysis is the speed-of-light (SOL) time: the shortest execution time permitted by a limiting hardware resource. For a memory-bound kernel, this limit is determined primarily by the peak memory bandwidth of the device:

$$t_{\text{SOL}} = \frac{D_{\text{model}}}{B_{\text{peak}}},$$

where D_{model} is the estimated amount of data moved and B_{peak} is the theoretical peak memory bandwidth. For a single seven-point stencil update, a simple global-memory traffic model assumes that each interior grid point requires:

- one read of the centre value,
- six reads of its spatial neighbours,
- one write of the updated value.

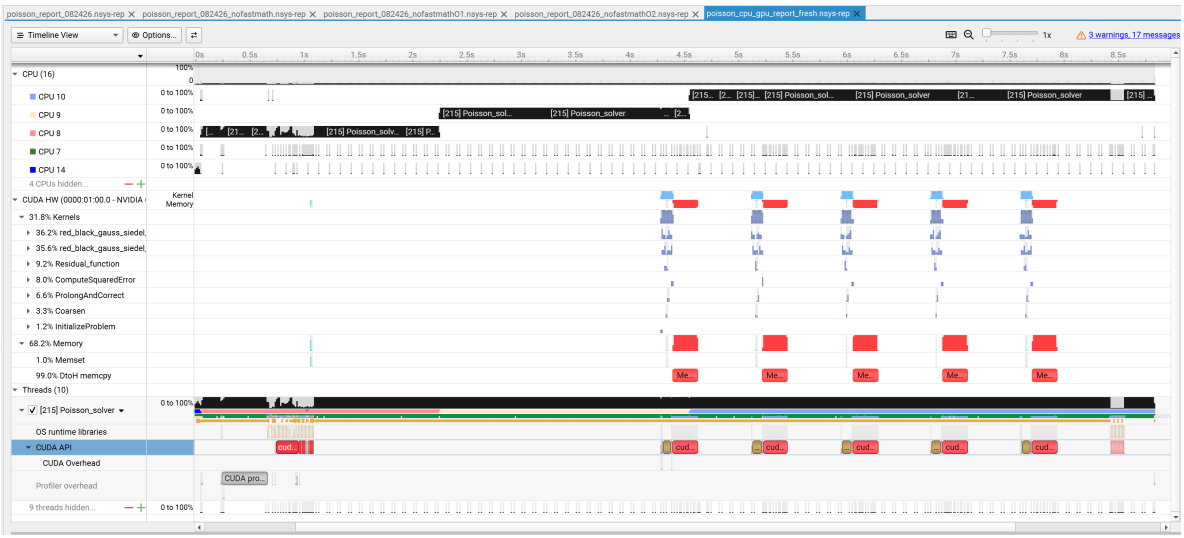


Figure 2: Poisson_basic.cu: Nsight timeline of the program. Here we can clearly see large gaps of GPU idle as it is waiting on data transfers; there is no overlap between compute and data transfer. The application is processing each phase linearly before copying data back or synchronizing, leaving wide white gaps where the GPU sits completely idle between iterations while waiting for the host thread to coordinate the next step. The solid red blocks under the memory activity bar indicate prolonged or frequent device memory transfer actions

#	Name	Start	TID
1	Starting level building	0.147601s	298
2	Level building complete	0.889032s	298
3	PROBLEM INITIALIZATION START	0.889037s	298
4	PROBLEM INITIALIZATION END	4.20218s	298
5	Execution loop Start	4.20221s	298
6	Execution loop Iteration	5.12533s	298
7	Execution loop Iteration	6.06235s	298
8	Execution loop Iteration	6.89266s	298
9	Execution loop Iteration	7.72108s	298
10	Execution loop Iteration	8.55144s	298
11	Execution loop End	8.55145s	298
12	MEMORY CLEANUP Start	8.55145s	298
13	MEMORY CLEANUP End	8.67054s	298

Figure 3: Poisson_basic.cu nvtx markers timestamps. Here we see that nearly half the runtime is spent on problem initialization (3.3s), but also see that initial launches take 0.92s and then decrease to 0.83s on subsequent loops.

Using single-precision floating-point values, the estimated data volume per grid point is therefore:

$$d_e = (1 + 6 + 1) \times 4 = 32 \text{ bytes.}$$

For the finest grid, with $N = 1025$ points per side, the total estimated data volume is:

$$D_{\text{model}} = N^3 d_e = 1025^3 \times 32 \approx 34.46 \text{ GB.}$$

Using the theoretical peak memory bandwidth of the RTX 5090:

$$B_{\text{peak}} = 1.792 \text{ TB/s} = 1792 \text{ GB/s,} \text{ [NVI26j]}$$

the corresponding speed-of-light time is:

$$t_{\text{SOL}} = \frac{34.46 \text{ GB}}{1792 \text{ GB/s}} \approx 1.92 \times 10^{-2} \text{ s} = 19.2 \text{ ms.}$$

If the measured kernel execution time is:

$$t_{\text{meas}} = 70 \text{ ms,}$$

then the achieved memory bandwidth is:

$$B_{\text{eff}} = \frac{D_{\text{model}}}{t_{\text{meas}}} = \frac{34.46 \text{ GB}}{0.070 \text{ s}} \approx 492 \text{ GB/s.}$$

The bandwidth utilisation is therefore:

$$\eta = \frac{B_{\text{eff}}}{B_{\text{peak}}} = \frac{t_{\text{SOL}}}{t_{\text{meas}}} \approx 0.275 = 27.5\%.$$

4.2 Compilation flags

The initial compilation command was

```
nvcc -O3 --use_fast_math --generate-code arch=compute_120,code=sm_120
--ptxas-options=-v Poisson_basic.cu -o Poisson_solver
```

The flags were selected as follows:

- `-O3` enables aggressive optimisation of both CPU and GPU code.
- `--use_fast_math` replaces certain IEEE-compliant floating-point operations with faster approximations, particularly for division, square roots, and trigonometric functions. This can improve performance but may reduce numerical precision [NVI].
- `--ptxas-options=-v` enables verbose `ptxas` output, reporting per-kernel register, shared, and constant memory usage. This helps identify register pressure and whether variables spill to local memory [NVI].
- `--generate-code arch=compute_120,code=sm_120` targets the Blackwell architecture by generating SASS for `sm_120`. Without an appropriate architecture specification, the driver may need to just-in-time compile PTX, adding load-time overhead.

The `--use_fast_math` flag is a high-level switch that enables several lower-level optimisations, including `-ftz=true`, `-prec-div=false`, and `-prec-sqrt=false`. These options respectively flush subnormal numbers to zero, replace correctly rounded division with a faster approximation, and disable IEEE-compliant rounding for square roots [NVI]. Since the solver is expected to be memory-bandwidth limited, with kernel execution times on the order of 10^{-6} s even under worst-case conditions, aggressive floating-point optimisation is unlikely to be the main performance bottleneck. The initial command is therefore more aggressive than necessary, and the potential accuracy penalty introduced by `--use_fast_math` is not justified by a clear performance benefit.

4.3 Nsight runs and Timeline analysis

Examination of the CPU and GPU timelines in Nsight Systems was performed to identify the stages that dominate the execution time. All timings reported in this subsection are full solver execution times in seconds, measured on the system described in the hardware and software section. Table 1 summarises the main configurations evaluated during the optimisation process.

The first 4.3 s are dominated by CPU activity, corresponding to memory allocation and calculation of the initial grid values; however, the `InitializeProblem()` kernel accounts for only 1.2% of the total GPU time, suggesting that this phase is unlikely to be the main bottleneck.

To delineate the major program stages more precisely, NVTX annotations (such as `nvtxMarkA()`) were inserted, which showed that level construction occurs from 148 ms to 889 ms, followed by problem initialisation until 4.20 s. The execution loop then runs from 4.20 s to 8.55 s, with the main V-cycle activity occurring between approximately 4.3 s and 8.0 s and iteration markers appearing at 5.12 s, 6.06 s, 6.89 s, and 7.72 s, after which memory clean-up runs from 8.55 s to 8.67 s. Enabling privileged mode in Docker allowed the CPU trace to be captured, and after modifying the zero-padding kernel and adopting an asynchronous pipelined approach, the measured runtimes were 1.830 s, 1.855 s, and 1.730 s, representing a substantial improvement over the previous baseline.

Table 1: Mean execution times for optimisation runs.

Run	Change	Mean (s)	Std (s)
1	Baseline	1.423	0.052
2	Better DiffSq	1.415	0.053
3	2D bdy + DiffSq	1.340	0.035
4	+ combining red-black kernel	1.378	0.055
5	+ residual	1.375	0.088
6	+ V-cycle	1.327	0.062
7	2D bdy + V-cycle	1.232	0.031
8	+ coarsen	1.340	0.175
9	New default	1.521	0.037
10	+ ProlongAndCorrect	1.394	0.094
P	Polynomial	1.495	0.058

4.4 Nsight Compute analysis

The new `InitializeProblem()` kernel performs substantially better than the original implementation. However, the coarsen kernel has a similar execution time in both versions, which suggests that the overall speed-up is not due solely to that change. A major limitation of the current solver is that the grid size decreases rapidly during the V-cycle. As the grid becomes smaller, there is less parallel work available, which can reduce GPU saturation. A future implementation could use larger or more consistent grid sizes in order to improve thread utilisation. The Thrust reduction kernel provides a useful reference because it is part of an optimised CUDA library. It achieves approximately 85% of both compute and memory throughput. In comparison, the final red-black kernel achieves approximately 67% compute throughput and 72% memory throughput in its best observed condition. The initial red-black kernel reaches up to 90% memory throughput, but its compute throughput is much lower, usually between 15% and 21%, and at most 32%. This suggests that the original kernel performs unnecessary memory transfers and leaves the compute units waiting for data.

Table 2: Nsight Compute throughput observations.

Kernel	Compute	Memory	Note
Thrust reduce kernel	85%	85%	Well-optimised reference
Initial red-black kernel	15-32%	max 90%	Low compute throughput
Final red-black kernel	67%	72%	Better balanced

5 Further Optimisation Attempts

5.1 Thread-block configuration

For the Blackwell target, several thread-block configurations were evaluated. A block size of `(32, 8, 4)` produced total runtimes of 8.849 s, 8.984 s, and 9.332 s, giving a mean runtime of 9.055 s. A block size of `(16, 16, 4)` produced runtimes of 8.351 s, 8.918 s, and 8.812 s, giving a mean runtime of 8.694 s. The `(16, 16, 4)` configuration gave consistently lower measured runtimes and was therefore used for the subsequent experiments.

5.2 Asynchronous V-cycle execution

A significant part of the optimisation effort was spent redesigning the V-cycle. The original implementation used a largely synchronous execution model, in which kernel launches and memory transfers were not sufficiently overlapped. In the Nsight Systems timeline, this appeared as short intervals of GPU activity separated by CPU idle gaps and synchronous memory-transfer stalls.
After introducing asynchronous kernel launches and stream-based memory transfers, the host enqueues work ahead of the device. In practice, the host remained approximately one or two launch steps ahead of the GPU, reducing idle gaps in the timeline and producing a more continuous stream of GPU activity. This improved utilisation of the RTX 5090 execution units and reduced the time spent waiting for synchronous transfers.

5.3 Major Changes to the code

Several major code changes were made to remove sources of inefficiency in the original implementation. In particular, the original version contained many unnecessary `cudaMemset()` calls during problem initialisation, suboptimal thread efficiency in the `red_black_gauss_seidel()` kernels, and unnecessary device-memory allocations. To aid performance analysis, NVTX ranges were added using `nvtxRangePushA()` and `nvtxRangePop()`. These annotations allowed individual kernels, memory transfers, and host-side operations to be identified more clearly in the Nsight Systems and Nsight Compute timelines. The boundary-initialisation logic was also revised. The new `ZeroBoundary3D()` routine zeroes only the six faces of each residual array `d_r`, rather than the full array. This initialisation is performed once and reduces the associated memory traffic by approximately a factor of 170 compared with zeroing the full array using `cudaMemset()`. The interior of `d_r` is overwritten by the residual kernel during each cycle.

5.4 Compiler and memory-hierarchy optimisations

Several features of the CUDA memory hierarchy and the NVCC compiler were used to improve the efficiency of the generated kernel code. Constant memory is a small read-only memory region that is cached on the GPU. It is most effective when many threads in a warp read the same address, because the hardware can broadcast the value to the whole warp. It is suitable for small immutable parameters, such as grid dimensions, physical constants, and stencil coefficients [NVI26g]. It is therefore not generally suitable for large arrays with highly divergent addresses, because non-uniform constant-memory accesses can be serialised.
The read-only data cache provides a separate read path to global memory. It can improve performance for immutable data with spatial or temporal locality, including some non-contiguous access patterns. Routing read-only array data through this path can reduce pressure on the general-purpose data cache and reduce cache thrashing. Invariant local variables are declared `const`. For example:

```
const size_t slice = fNx * fNy;
```

tells the compiler that the value is immutable. This enables constant propagation, removes redundant re-computation of index expressions, and can reduce register pressure by allowing the compiler to reason more precisely about the value's lifetime. In some cases, the value can be folded directly into subsequent instructions. Large read-only arrays are passed using `const __restrict__` pointers. For example:

`const float __restrict__ *fineGrid`

provides two guarantees to the compiler: the data is not modified through that pointer, and the pointer does not alias other pointer arguments [NVI26f]. These guarantees allow NVCC to select more efficient load paths, including the read-only data cache. The `__ldg()` intrinsic can also be used to request an explicit read-only global load [NVI26h].

5.5 Limitations and future work

The main limitation of the present evaluation is that the multigrid grid size decreases rapidly across levels (halves in size on each coarsening step). On the coarser grids, the GPU is not fully utilised, which limits the observable benefit of some kernel optimisations. In addition, the timing measurements showed non-negligible run-to-run variability, so small differences between configurations should not be over-interpreted. A future project could improve on this work by using a larger and more consistent grid size, benchmarking each multigrid level separately, or applying more detailed profiling to determine whether the remaining bottleneck is memory latency, occupancy, thread underutilisation, or host–device transfer overhead. The comparison with the Thrust reduction kernel suggests that there is still room for improvement in the memory access patterns and data movement efficiency of the custom kernels.

5.6 Changes made

5.6.1 New kernel: `multigrid_red_black_fused()`

The main smoothing kernel, `multigrid_red_black_fused()`, performs both red and black Gauss–Seidel updates within a single kernel launch. This reduces kernel-launch overhead compared with launching separate red and black kernels. Grid points are classified as red or black using the parity of their three-dimensional coordinates:

$$(x + y + z) \bmod 2.$$

Within each colour class, the updated grid points are independent, so the corresponding updates can be executed in parallel. After the red update, a synchronisation barrier ensures that the updated values are visible before the black update is performed. The kernel loads the local tile, together with its halo, into a padded shared-memory array:

```
__shared__ float s_u[BLOCK_Z + 2][BLOCK_Y + 2][BLOCK_X + 2];
```

The additional one-cell padding in each direction provides halo, or ghost, cells. This allows threads on the boundary of the thread block to access neighbouring values without triggering out-of-bounds shared-memory accesses and without requiring conditional boundary checks in the inner stencil loop. Synchronisation barriers, implemented using `__syncthreads()`, ensure that the shared-memory tile and halo are completely loaded before any stencil updates are performed.

5.6.2 Changes in the revised `ProlongAndCorrect()`

The revised `ProlongAndCorrect()` kernel preserves the original trilinear prolongation and in-place correction, but changes several implementation details.

- The thread coordinates are now declared as `const unsigned int` instead of `size_t`. The z-coordinate is also renamed from `z` to `fz` for consistency with the fine-grid indexing.
- The boundary guard now uses explicit unsigned comparisons of the form `fx == 0` rather than `fx <= 0`. For unsigned indices, the two forms are equivalent, but the revised form is clearer.
- The mapping from fine-grid coordinates to coarse-grid coordinates now uses a right shift,

  $$c_x = f_x \gg 1,$$

  instead of integer division by two. For non-negative integer coordinates, this is equivalent to $\lfloor f_x/2 \rfloor$ and may be slightly cheaper.

- The helper function `getLinearIdx3D()` is no longer used. The coarse and fine linear indices are now computed explicitly inside the kernel. By switching to using read-only load intrinsic, the compiler/hardware is allowed to use the read-only data path
- The coarse-index arithmetic is reorganised by pre-computing the z-plane offsets and y-row strides:

$$z_0 = c_z^0 N_x^c N_y^c, \qquad y_0 = c_y^0 N_x^c,$$

  with analogous quantities for the upper coarse indices. This reduces repeated index calculations when reading the eight surrounding coarse-grid values.
- The eight coarse-grid reads now use the `__ldg()` intrinsic, which directs the loads through the read-only data path/cache. The original kernel used ordinary global-memory reads.
- The fine-grid index is now computed explicitly using 64-bit arithmetic:

$$i_f = f_z N_x^f N_y^f + f_y N_x^f + f_x.$$

- The final update remains an in-place correction:

$$u_f \leftarrow u_f + P u_c.$$

  The trilinear interpolation formula is unchanged.

Overall, the revised kernel is algebraically equivalent to the original, but it uses more explicit 64-bit indexing, 32-bit unsigned thread coordinates, shift-based coarse indexing, and read-only cache hints for the coarse-grid loads.

5.6.3 Changes in the revised `v_cycle_hierarchical()`

The revised `v_cycle_hierarchical()` routine preserves the same recursive V-cycle structure, but changes the CUDA execution strategy. The most important differences are as follows.

- A CUDA stream argument, `cudaStream_t stream`, has been added to the function interface. All kernel launches and the coarse-grid memory clear are now enqueued on this stream. This allows the V-cycle to be integrated into an asynchronous execution pipeline.
- The separate red and black Gauss–Seidel kernel launches are replaced by a single fused red–black smoothing kernel, `multigrid_red_black_fused`. Consequently, each smoothing iteration now requires one kernel launch instead of two.
- Two launch configurations are now used. The smoothing kernel uses an adaptive `block_fused` configuration, based on $16 \times 8 \times 4$ threads when the grid is large enough in each direction. The residual, restriction, and prolongation kernels retain the previous legacy configuration, `block_legacy`, based on $32 \times 8 \times 1$ threads.
- The synchronous call `cudaMemset` used to initialise the coarse-grid solution has been replaced by `cudaMemsetAsync`, enqueued on the supplied CUDA stream. This removes a host-side blocking operation from the V-cycle path.
- The smoothing and residual kernels now receive only h^2, rather than both h and h^2. The host function still computes the coarser grid spacing $h_{\ell+1} = 2h_\ell$ for the recursive call, but the kernels no longer require h directly.
- At the coarsest level, the additional smoothing loop is annotated with `#pragma unroll`. The loop still performs ten smoothing iterations, but the host-side loop overhead is reduced.
- The recursive call now passes the CUDA stream to the coarser-level V-cycle, so the entire multigrid recursion is submitted through the same stream.

Overall, the revised driver reduces kernel-launch overhead for smoothing, uses a more architecture-aware launch configuration for the smoother, and makes the V-cycle execution asynchronous through explicit stream management. The mathematical sequence of the V-cycle remains the same.

5.6.4 Changes in the revised `Coarsen()`

The revised `Coarsen` kernel differs from the original in the following ways.

- Thread indices are now declared as `const unsigned int` rather than `size_t`. This helps in the process of compiler optimisation, by declaring these as non changing during its scope [NVI26c]. It helps nvcc resolve pointer aliasing (whether two pointers point to the same location).
- The fine-grid anchor is computed using a left bitwise shift, `x << 1`, instead of multiplication by two. Less computation is required, as fewer instructions are needed to obtain the result.
- The helper function `getLinearIdx3D` is no longer used. Fine-grid and coarse-grid linear indices are computed directly, and are declared as const for better compiler optimisations. The stencil weights are declared as compile-time constants in the interior path. The fine-grid slice size, `fNx * fNy`, is precomputed once.
- The 27-point stencil is split into an interior path and a boundary path. The interior path is manually unrolled and avoids runtime `min`/`max` clipping operations. The interior path uses explicit stencil offsets grouped by $z = -1$, $z = 0$, and $z = +1$ planes. Boundary stencil points are skipped rather than clamped.
- Fine-grid reads use load-from-global, `__ldg()` (`ld.global.nc`), in both the interior and boundary paths. `__ldg()` performs a read-only L1/Tex cache load [NVI26d]. Because it signals to the GPU hardware that a global memory data load is strictly read-only, it bypasses standard L1 logic for reads, thus reducing cache pollution in the primary data cache and significantly reducing instruction latency for non-coalesced or irregular access patterns [NVI26l].
- The boundary path rescales the accumulated weighted sum by the total weight of the valid stencil contributions. `#pragma unroll` instructs the compiler to completely flatten the loops at compile time, because the loop bounds are small and statically known [NVI26e]. Through exposing all 27 memory reads and math operations sequentially, the compiler can intelligently reschedule instructions. Instead of a strict "read-then-multiply" block-by-block execution, the NVCC compiler can pipeline the operations: it can issue multiple `__ldg()` global memory reads back-to-back to hide latency, and then perform the math operations (sum += ...) as the data arrives from the read-only cache.

Stencil comparison

Let F denote the fine-grid values and let C_{ijk} denote the coarse-grid value associated with fine-grid anchor $(2i, 2j, 2k)$. The stencil weights depend on the Manhattan distance from the centre:

$$w(a,b,c) = \begin{cases} 1/8, & |a|+|b|+|c| = 0, \\ 1/16, & |a|+|b|+|c| = 1, \\ 1/32, & |a|+|b|+|c| = 2, \\ 1/64, & |a|+|b|+|c| = 3. \end{cases}$$

In the original kernel, out-of-bounds stencil points were clamped to the nearest valid fine-grid cell. The coarse value was therefore

$$C_{ijk} = \sum_{a,b,c=-1}^{1} w(a,b,c) F_{\mathrm{clip}(2i+a),\mathrm{clip}(2j+b),\mathrm{clip}(2k+c)},$$

where clip restricts each index to the valid fine-grid range. For interior points, the revised kernel applies the same stencil without clamping:

$$C_{ijk} = \sum_{a,b,c=-1}^{1} w(a,b,c) F_{2i+a,\,2j+b,\,2k+c}.$$

For boundary points, the revised kernel omits out-of-bounds contributions. If S_{ijk} denotes the set of valid stencil offsets and $W_{ijk} = \sum_{S_{ijk}} w(a,b,c)$, then the revised boundary update, as implemented, is:

$$C_{ijk} = \frac{1}{8W_{ijk}} \sum_{(a,b,c)\in S_{ijk}} w(a,b,c) F_{2i+a,\,2j+b,\,2k+c}.$$

5.6.5 Changes in the revised `Residual_function()`

The revised kernel preserves the same mathematical residual but makes the implementation more explicit and improves the clarity of the stencil evaluation. The main changes are as follows.

- The unused argument h was removed from the kernel interface. The revised kernel only receives h^2, since only h^{-2} is required for the stencil.
- The thread indices are now declared as `const unsigned int` rather than `size_t`. This makes the unsigned nature of the CUDA thread indices explicit.
- The helper function `getLinearIdx3D()` was replaced by a direct computation of the linear index:

  $$i = zN_xN_y + yN_x + x,$$

  with the stride N_xN_y computed once. This makes the memory layout visible directly in the kernel.
- The neighbour values are accumulated in a separate sum before applying the seven-point operator. This separates the stencil sum from the final residual update and makes the code easier to read.
- Reads of u and f are performed using `__ldg()`, which directs the loads through the read-only data path/cache. This is appropriate for stencil reads where the data are read-only and may be shared among threads.
- The stencil is written in the form:

  $$Au_i = \left(6u_i - \sum_{j\in\mathcal{N}(i)} u_j\right) h^{-2},$$

  which is algebraically identical to the original expression but avoids a potentially confusing sign in the intermediate variable.
- Verbose comments and commented-out code were removed, leaving only short comments describing the major steps of the kernel.

The mathematical update remains:

$$r_i = f_i - Au_i.$$

The revised kernel therefore produces the same residual as the initial implementation, while using a cleaner interface, more explicit indexing, and more explicit read-only memory loads.

5.6.6 Changes in the revised `main()`

There are many similarities between the original `main()` function and the revised one. They both initialise the starting grid of Nx = Ny = Nz = N = 1025, h = L / (Nx - 1), h2 = h * h, and then both set the target limit of target_coarse_limit = 33. Both versions build the multigrid hierarchy using the same vertex-centred coarsening rule: current_N = ((current_N - 1) / 2) + 1. The base mathematical setup is therefore unchanged, and the differences are in how the program is driven, timed, and verified. Both versions print the dimensions and approximate memory footprint for each multigrid level. They print Nx_lvl[l], Ny_lvl[l], and Nz_lvl[l], compute the size in megabytes for one array, and then multiply by 3 to report the combined footprint of u, f, and r.

- Removal of full hierarchy zeroing, instead relying on the `ZeroBoundary3D()` kernel to initialize or overwrite the arrays. for a large grid such as 1025^3, zeroing all arrays can involve many gigabytes of memory traffic. We now avoid performing unnecessary writes, because of the massive penalty as evidenced in the Nsight timeline. The old version allocates and zeroes all arrays at every level. For each level, it allocates: `d_u_ptr`, `d_f_ptr` and `d_r_ptr`. Then it immediately calls `cudaMemset(d_u_ptr, 0, total_bytes)`, `cudaMemset(d_f_ptr, 0, total_bytes)` and `cudaMemset(d_r_ptr, 0, total_bytes)`. It writes zero to every element of every u, f, and r

array, and for a large 3D grid, this can be a significant memory-bandwidth cost. The new version allocates the same arrays but does not call `cudaMemset()`, only `cudaMalloc()`. The zeroing is now done at: `d_u[0]` is zeroed by `InitializeProblem()` `d_u[l > 0]` is zeroed by the per-cycle logic inside `v_cycle_hierarchical()` `d_f[*]` is overwritten by initialization or coarsening `d_r[*]` boundaries are zeroed by `ZeroBoundary3D()`, and interiors are rewritten each cycle.

- Error computation moves from host to GPU. We want to reduce the amount of work the CPU does, as it will be less efficient, whilst avoiding constant kernel launches.

- Dedicated CUDA stream and event-based timing Old version used `nvtxMarkA()` to mark significant events in the execution sequence. Whilst helpful initially, this is very limited in its useful scope, and is less convenient for profiling. New version uses `nvtxRangePushA()` and `nvtxRangePop()`, which give ranges that show the duration of the phases, and are easier to inspect with Nsight tools, and give clarity on which parts of the program are being timed. In addition, the new version creates performance-measurement objects; A CUDA stream: mg_stream; A start event: start_event; A stop event: stop_event; A device scalar: d_sum; A host scalar: h_sum; and CUB temporary storage: d_temp_storage. The old version did not perform per-cycle timing.

- Change in measurement metric: RMS vs L2 norm The error is measured using the root-mean-square error

$$\mathrm{RMS}(e) = \left(\frac{1}{N^3} \sum_i \left(u_i - u_i^{\mathrm{true}} \right)^2 \right)^{1/2}.$$

  The error is now measured using the discrete L_2 norm

$$\|e\|_2 = \left(h^3 \sum_i \left(u_i - u_i^{\mathrm{true}} \right)^2 \right)^{1/2},$$

  where the sum is taken over all finest-grid points.

- The new version adds a warmup kernel: `warmup()¡¡¡1, 1¿¿¿`, which absorbs the one-time costs before the timed solver loop, and make subsequent measurements more reliable.

5.6.7 Changes in utility functions and macros

- `abs_dx_calc()` computes the Manhattan distance, or L_1 norm, of a difference vector. The function is marked `__forceinline__`, which encourages the compiler to inline the calculation directly into the calling kernel and avoid function-call overhead in performance-critical GPU code.

- `DifferenceSquared` is a struct using thrust library functions to compute the squared difference between paired floating-point values. It is used in reductions and error calculations where many differences are accumulated. The functor performs the calculation in double precision to reduce the accumulation of rounding errors over large numbers of operations. A possible optimisation is to cast the operands to `double` before subtraction, rather than only before squaring. This may increase register pressure, because double-precision values require 64 bits, but it can improve numerical accuracy.

- `ZeroBoundary3D()` zeroes the boundary cells of a three-dimensional array. It is used once during start-up to initialise residual arrays. An alternative implementation would use six separate two-dimensional kernels, one for each face of the domain. This may improve memory coalescing and reduce register pressure, although it would increase the number of kernel launches.

- `warmup()` performs a one-time warmup launch before timing begins. This absorbs first-launch overhead, such as driver initialisation and module loading, so that this cost is not included in the measured timing.

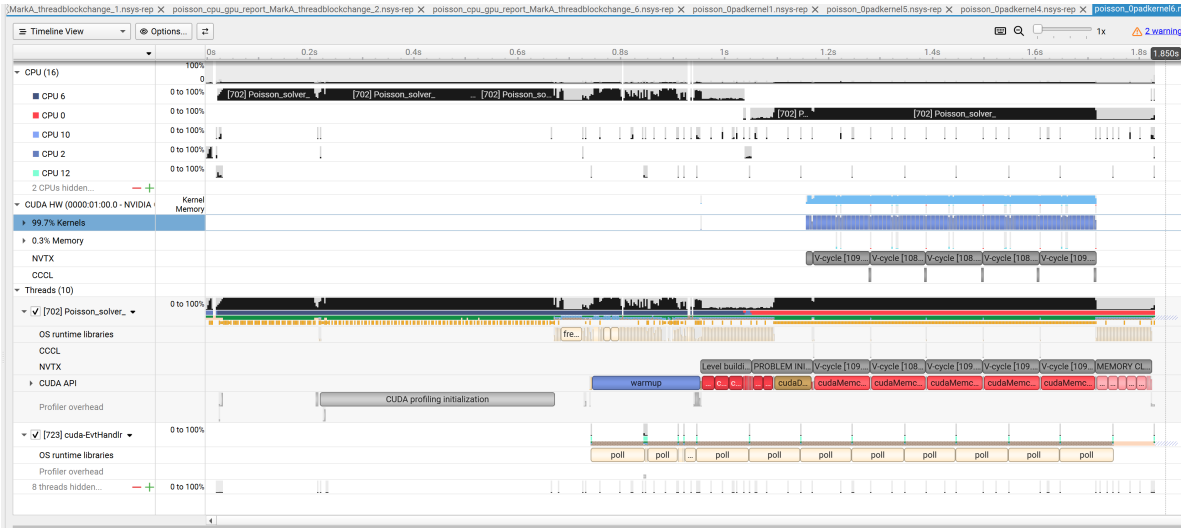


Figure 4: Poisson_redesign_final.cu: Here we can clearly see the improvements to the timeline. Initialisation phase now concludes in roughly 0.25 seconds on this specific timeline segment. The long, drawn-out idle gaps on the GPU are completely gone The kernels here are packed tightly back-to-back. The CPU is aggressively queuing and launching kernels, completely hiding the launch latencies. The bottom rows under the active compute sections, you can see a dense stream of red blocks indicating memory interactions or synchronization tokens during the V-cycle iterations. The V-cycles run back-to-back so rapidly, this confirms that the math execution completely dominates the timeline footprint during the computation window.

6 Conclusions

Prior to this work, I was largely unfamiliar with the extensive ecosystem of auxiliary libraries and hardware intrinsics available to dramatically accelerate solver efficiency. Driven by a motivation to understand the finer nuances of GPGPU computing—including floating-point handling and compiler optimization mechanics, I was able to transition theoretical concepts into a highly optimized, high-performance implementation.

The architectural contrast between the CPU and GPU highlights why this transition requires strict algorithmic discipline. A single CPU core dedicates the vast majority of its silicon real estate to control logic and hardware helpers. Features such as multi-megabyte cache hierarchies to eliminate memory wait times, out-of-order execution pipelines to schedule independent commands ahead of time, and sophisticated branch predictors all serve to cushion the performance impact of irregular programming structures. This inherently grants the developer significantly more structural leeway and flexibility [HP19]. Conversely, a GPU core strips away almost all of this administrative overhead. It eschews complex branch prediction, lacks out-of-order execution logic, and possesses only a minimal local cache structure in order to pack the silicon almost entirely with mathematical Arithmetic Logic Units (ALUs).

While I had previously understood that the GPU's shallower cache hierarchy and longer memory latency impose strict execution constraints, putting this theory into practice revealed the true rigour of the programming model. CUDA C++ forces the programmer to explicitly manage heterogeneous, isolated physical and virtual address spaces through strict orchestration APIs like cudaMalloc and cudaMemcpy. To maximize throughput, data must be manually partitioned into specialized structures such as ultra-fast, software-managed shared memory, constant memory, or pulled through dedicated read-only texture caches via intrinsics like __ldg(). Engineering these low-level memory pathways within a mathematically dense stencil solver was a challenging but vital learning experience.

Through systematic profile-driven optimizations, a substantial efficiency boost was achieved, reducing total application execution time from over 8.6 seconds to less than 1.9 seconds. Profiling revealed inherent architectural limitations to further optimization; namely, the algorithmic nature of a multigrid solver leads to progressively lower GPU utilization at the coarsest grid levels. However, securing a 75% overall reduction in runtime demonstrates that plenty of optimization opportunities were successfully

exploited. Ultimately, this project served as an enjoyable exercise in conquering an unfamiliar domain, vastly increasing my confidence in writing, analysing, and effectively benchmarking accelerated code. Figure 4 shows the timeline of the final version of my code, and clearly indicates how much the program has evolved. There was was a clear advantage to taking advantage of the compiler specific optimisation and restructuring the algorithm, however these must be paired with micro-architectural awareness to achieve peak performance. By leveraging `__forceinline__`, `__restrict__` qualifiers and `__ldg()` intrinsics, the compiler successfully bypassed conservative aliasing assumptions, maximizing read-only data cache utilization. Hardware-specific optimizations yielded substantial performance dividends, proving that managing register pressure and cache behaviour can significantly reduce runtimes - which can in turn allow for more iterations in a given time or faster results. This project provided a deeply rewarding opportunity to explore the application of CUDA C++ in solving complex partial differential equations on a 3D domains. Overall, this was a very interesting project for me to work on, and I am excited to see what I can develop next.

References

[AWH13] George B. Arfken, Hans J. Weber, and Frank E. Harris. Chapter 9 - partial differential equations. In *Mathematical Methods for Physicists (Seventh Edition)*, pages 401–445. Academic Press, Boston, seventh edition edition, 2013.

[BH12] Nathan Bell and Jared Hoberock. Thrust: A productivity-oriented library for CUDA. In *GPU Computing Gems Jade Edition*, pages 359–371. Morgan Kaufmann, 2012.

[BHM00] William L. Briggs, Van Emden Henson, and Steve F. McCormick. *A Multigrid Tutorial*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, second edition, 2000.

[HP19] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.

[Hun] R E Hunt. Chapter 2 poisson's equation. https://www.damtp.cam.ac.uk/user/reh10/lectures/nst-mmii-chapter2.pdf.

[KHEH22] David B. Kirk, Wen-mei W. Hwu, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, fourth edition, 2022.

[MN24] Duane Merrill and NVIDIA Corporation. Cub: Cuda unbound, 2024.

[NVI] NVIDIA Corporation. nvcc compiler switches : Cuda c++ best practices guide 12.6 documentation. https://docs.nvidia.com/cuda/archive/12.6.1/cuda-c-best-practices-guide/nvcc-compiler-switches.html [Accessed: 2026-06-28].

[NVI26a] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26b] NVIDIA Corporation. *CUDA C++ Programming Guide, 2.3.3.2. Shared Memory*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26c] NVIDIA Corporation. *CUDA C++ Programming Guide, 5.3.10.4.2. const*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26d] NVIDIA Corporation. *CUDA C++ Programming Guide, 5.4.8.3. Low-Level Load and Store Functions*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26e] NVIDIA Corporation. *CUDA C++ Programming Guide, 5.4.9.1. pragma unroll*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26f] NVIDIA Corporation. *CUDA C++ Programming Guide, Section 10.2.6.* NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26g] NVIDIA Corporation. *CUDA C++ Programming Guide, Section 2.3.3.5.* NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26h] NVIDIA Corporation. *CUDA C++ Programming Guide, Section 5.4. C/C++ Language Extensions*. NVIDIA Corporation, 2026. Available at https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf, Last accessed: September 2026.

[NVI26i] NVIDIA Corporation. CUDA Toolkit Documentation, 2026. Version 13.4, Accessed: September 2026.

[NVI26j] NVIDIA Corporation. NVIDIA GeForce RTX 5090 Graphics Cards. https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5090/, 2026. Accessed: September 28, 2026.

[NVI26k] NVIDIA Corporation. *NVTX (NVIDIA Tools Extension Library) Documentation*. NVIDIA Corporation, 2026. Available at https://nvidia.github.io/NVTX/, Last accessed: September 2026.

[NVI26l] NVIDIA Corporation. *Parallel Thread Execution (PTX) ISA, 9.7.9.9 Data Movement and Conversion Instructions: ld.global.nc and 9.7.9.8 Data Movement and Conversion Instructions: ld*. NVIDIA Corporation, version 13.3 edition, 2026.

[OR10] William L. Oberkampf and Christopher J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, Cambridge, UK, 2010.

[PTVF07] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. Cambridge University Press, USA, 3 edition, 2007.

[Saa03] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, second edition, 2003.

[WL] Zilu Wang and Steve Lantz. Cornell virtual workshop : Introduction to cuda : Gpu performance topics : Coalesced memory access. https://cvw.cac.cornell.edu/cuda-intro/gpu-performance-topics/coalesced [Accessed: 2026-07-03].